SIMATS ENGINEERING

ASSESSMENT TOOL 1

Course Code	CSA04
Course Name	Operating Systems
Course Outcome	CO1: Apply the concepts of Operating System types, structures, and system calls to demonstrate their functions in various computing environments.
Questions / Marks	5 Questions / Total Marks: 100

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I **M. VENKATESH / 192511133** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Student Name	M.VENKATESH
Register Number	192511133

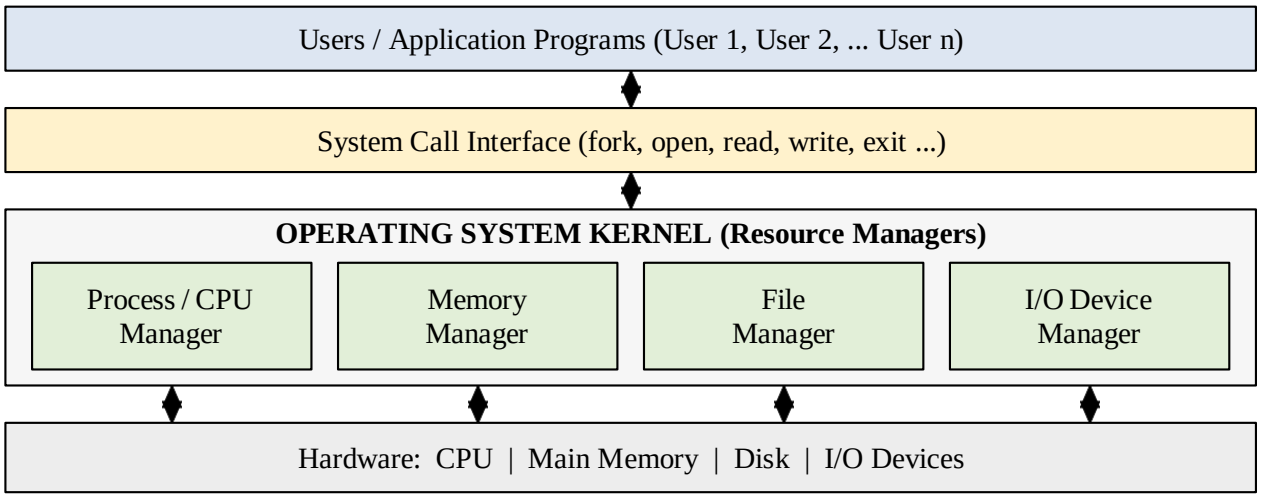
Question 1 (5 Marks)

(a) Types of Operating Systems suitable for the scenario

Multi-user Time-Sharing OS (Unix/Linux, Windows Server): the CPU is switched among users in small time slices, so every user feels the system is dedicated to them. **Multiprogramming / Multitasking OS**: several processes stay in memory so the CPU is never idle during I/O. **Multiprocessor (Parallel) OS**: shares CPU and memory among many processors for higher throughput.

Justification: Many users and processes compete for the same CPU, memory and I/O devices. A time-sharing multiprogramming OS provides fair CPU scheduling, memory protection between users, device spooling and good response time, which is exactly what this shared environment needs.

(b) Structure of the Operating System and resource management



Each manager in the kernel controls one resource: CPU (process manager), RAM (memory manager), disk files (file manager) and devices (I/O manager).

(c) System call sequence to create and execute a new process

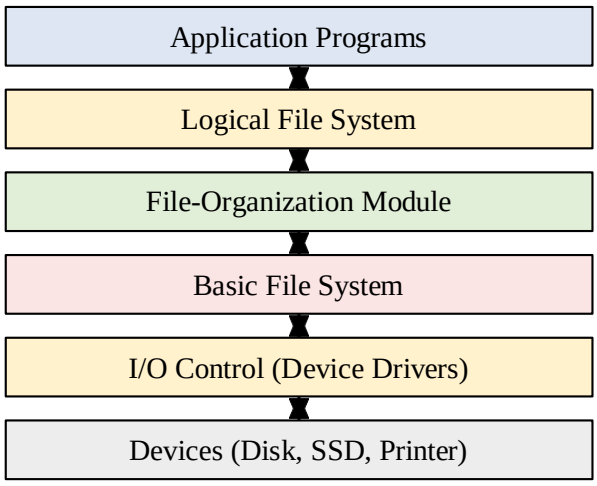
```
/* 1. create child process */
pid = fork();
/* 2. child loads a new program */
if (pid == 0) execlp("/bin/ls", "ls", NULL);
/* 3. parent waits for the child */
else wait(NULL);
/* 4. process terminates */
exit(0);
```

Question 2 (5 Marks)

(a) Categories of system calls for file and device management

Category	System calls
File management	create(), open(), read(), write(), lseek() (reposition), close(), unlink() (delete), get/set file attributes (stat, chmod)
Device management	request/release device, ioctl() (control), read(), write(), reposition, get/set device attributes, logically attach/detach devices

(b) Layered structure showing file operations



A call such as read() travels down the layers: the logical layer finds the file, the organization module maps the block, and the driver talks to the device; data returns upward.

(c) C system call sequence to open, read and close a file

```
#include <fcntl.h>
#include <unistd.h>
int fd = open("data.txt", O_RDONLY); /* open */
if (fd < 0) { perror("open"); return 1; }
char buf[100];
ssize_t n = read(fd, buf, 100); /* read */
write(1, buf, n); /* display on screen */
close(fd); /* close */
```

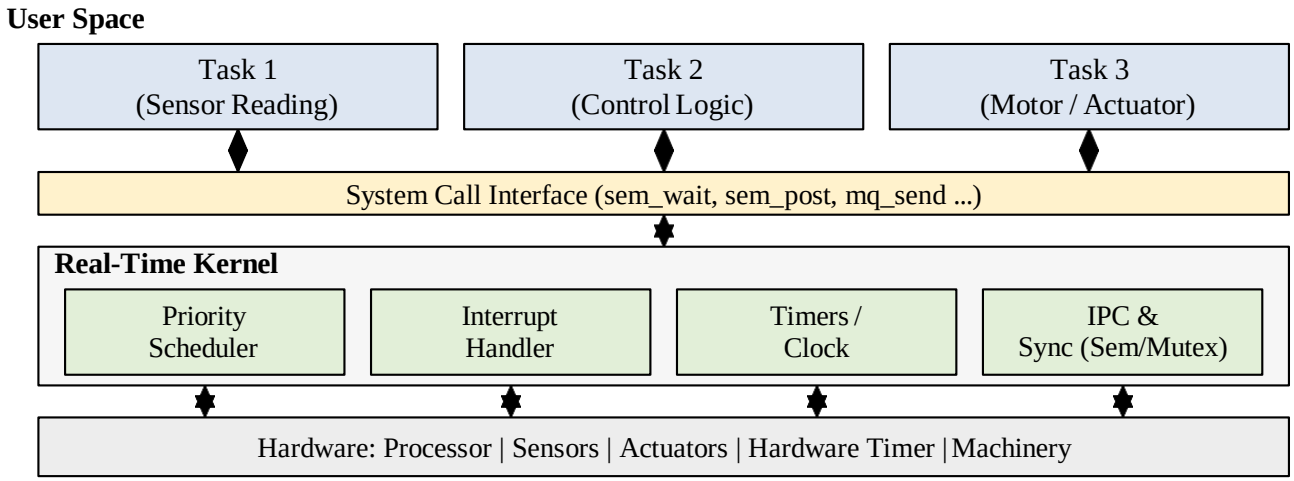
Question 3 (5 Marks)

(a) Type of OS and why it suits real-time applications

The system needs a **Hard Real-Time Operating System (RTOS)**, for example VxWorks, QNX or FreeRTOS. It is suitable because:

- it is **deterministic**: every task is guaranteed to finish before its deadline;
- it uses **priority-based preemptive scheduling**, so critical control tasks run first;
- it has **very low interrupt and context-switch latency** and a small, fast kernel;
- a missed deadline in industrial machinery can cause damage or danger, so timing is the key requirement.

(b) Interaction between hardware, kernel and user processes



(c) System calls for process synchronization

System call	Explanation
sem_init() / sem_destroy()	create and delete a semaphore with an initial count
sem_wait()	decrement semaphore; task blocks if resource is unavailable (P operation)
sem_post()	increment semaphore and wake a waiting task (V operation)
pthread_mutex_lock() / unlock()	give mutual exclusion over a shared machine resource
mq_send() / mq_receive()	message queue used to signal and pass data between tasks

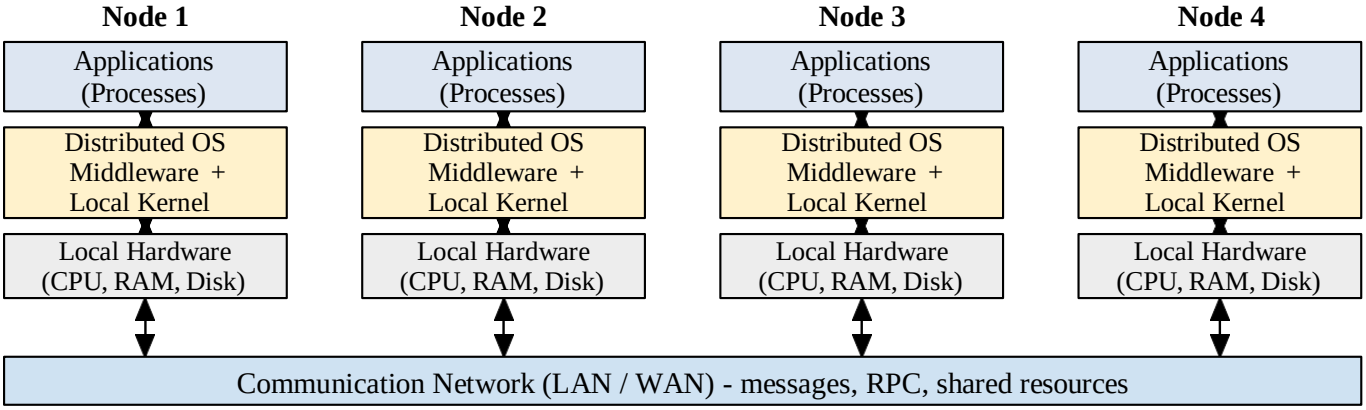
Question 4 (5 Marks)

(a) Type of OS and comparison with centralized OS

The system uses a **Distributed Operating System**, which makes many networked computers behave like a single system.

Feature	Centralized OS	Distributed OS
Control	One computer, one kernel	Many nodes, cooperating kernels
Resources	Local to one machine	Shared across the network
Failure	Single point of failure	Fault tolerant, others continue
Scalability	Limited by one machine	Add nodes easily
Communication	Shared memory, local IPC	Message passing, RPC

(b) Architecture of a distributed OS



(c) System calls for inter-process communication (message passing)

Server: socket() → bind() → listen() → accept() → recv() / send() → close()

Client: socket() → connect() → send() / recv() → close()

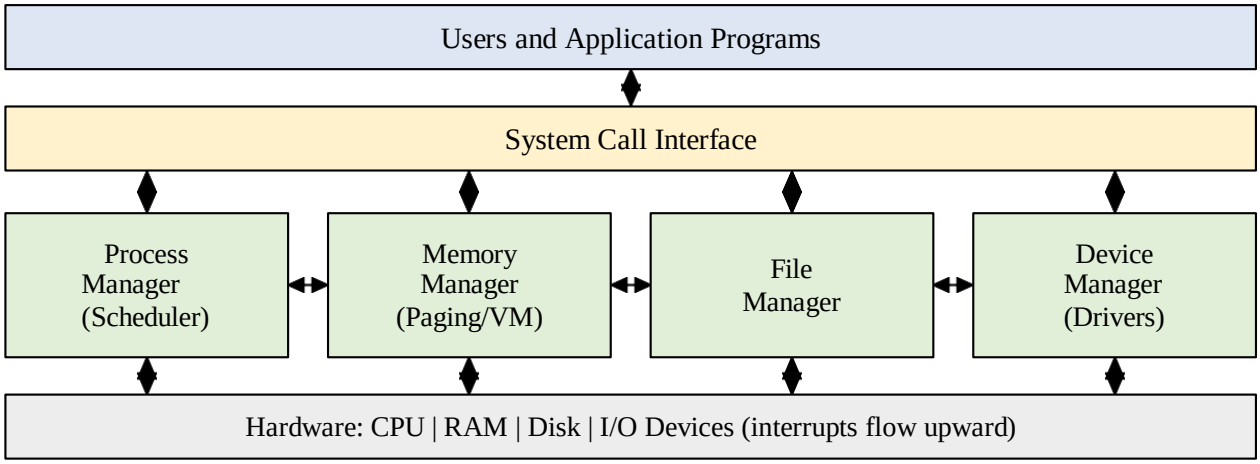
For processes on the same node, pipe(), shmget()/shmat() (shared memory) or msgget()/msgsnd()/msgrcv() are used instead.

Question 5 (5 Marks)

(a) Major functions of an OS

- **Process scheduling:** keeps a PCB for every process, maintains ready/wait queues, selects the next process (FCFS, SJF, Round Robin, Priority) and performs context switching so the CPU stays busy.
- **Memory allocation:** tracks free and used memory, allocates and frees it using paging/segmentation, provides virtual memory and protects one process from another.
- **Device communication:** device drivers hide hardware details, interrupts signal completion, and buffering, caching and spooling make I/O efficient.

(b) Block diagram of OS components and interactions



The managers cooperate: a process waiting for I/O is blocked by the process manager, and a device interrupt makes it ready again.

(c) System call examples

Task	System call examples
Memory allocation	malloc() / brk() / sbrk() to get memory, mmap() to map memory, munmap() and free() to release it
Device management	open("/dev/tty"), ioctl(fd, cmd, arg), read(fd, buf, n), write(fd, buf, n), close(fd)

```
char *ptr = (char *) malloc(100); /* allocate memory */
int fd = open("/dev/tty", O_RDWR); /* open device */
write(fd, "Hello", 5); /* write to device */
close(fd); free(ptr); /* release both */
```